

Open-Set Individual Bat Face Recognition: Identity-Disjoint Evaluation, Pair vs. Embedding Architectures, and the Small-Identity Generalisation Wall

Sagi Levi

lesagi@gmail.com

May 2026

Abstract

We present an open-set individual face recognition system for fruit bats, designed to support longitudinal behavioural studies where every animal must be identified without prior appearance in the training set. Our primary methodological contribution is a rigorous *identity-disjoint* evaluation protocol: train, validation, and test splits are partitioned at the level of individual identities so that no bat ever appears in more than one split. We show that the widely-used alternative—random pair selection—inflates performance dramatically: a Siamese network that achieves $F_1 = 0.99$ under the random-pair protocol drops to $F_1 = 0.74$ under identity-disjoint evaluation on the same images.

We compare three deep-learning model families—a pair-based Siamese convolutional network and two embedding-based angular-margin architectures (ArcFace and AdaFace built on a ResNet-50 backbone)—across a factorial design covering two fruit-bat species (*rousettus* and *mauritus*), three background conditions (green screen, original, and random compositing), and controlled identity counts. On the *rousettus* dataset (12 individuals, 1 059 images), the Siamese network achieves a test ROC-AUC of 0.809 versus 0.541 and 0.507 for ArcFace and AdaFace respectively. On *mauritus* (11 individuals, 1 119 images) the ordering is preserved, with the Siamese network reaching 0.803–0.925 depending on background condition. We trace the embedding-model shortfall to the *small-identity generalisation wall*: with fewer than twelve training identities the angular separation learned during closed-set training does not transfer to unseen test identities, even when training accuracy approaches 100%.

All experiments are fully reproducible via a single-command pipeline and are tracked in MLflow.

Contents

1	Introduction	4
2	Related Work	5
2.1	Animal re-identification	5
2.2	Metric learning and face verification	5
2.3	Angular margin softmax losses	5
2.4	Biometric evaluation metrics	6

3	Dataset	6
3.1	Species and capture conditions	6
3.2	Preprocessing pipeline	6
3.3	Manifest structure	6
3.4	Identity-disjoint splitting	7
3.5	Dataset statistics	7
4	Methods	7
4.1	Pair-based: Siamese Convolutional Network	8
4.2	Embedding-based: ArcFace and AdaFace	8
4.3	Hyperparameter tuning	9
5	Evaluation Protocol	9
5.1	Gallery and probe construction	9
5.2	Verification	9
5.3	Identification	11
5.4	Statistical significance: permutation test	11
6	Experiments and Results	12
6.1	Leaky vs. honest evaluation	12
6.2	Model family comparison on rousettus	12
6.3	Model family comparison on mauritius	12
6.4	ArcFace hyperparameter sensitivity	13
6.5	Learning curves	13
6.6	Family and background comparison (summary bar chart)	13
6.7	Reproducibility	13
7	Discussion	14
7.1	Why pair-based models win at small identity counts	14
7.2	Background effects	15
7.3	The leaky-vs-honest gap	15
7.4	Limitations	16
8	Conclusion	16
A	Software Architecture	19
A.1	Workspace layout	19
A.2	Dependency graph and invariants	19
A.3	Automated post-training pipeline	20

A.4	PDF report structure	21
B	Technology Stack	21
B.1	Core framework	21
B.2	Model and training infrastructure	21
B.3	Configuration management	21
B.4	Experiment tracking	21
B.5	Hyperparameter optimisation	22
B.6	Preprocessing	22
B.7	Interpretability and reporting	22
B.8	CLI surface	23
B.9	Quality assurance	23

1 Introduction

Fruit bats play pivotal roles in tropical and subtropical ecosystems as pollinators and seed dispersers, yet the individual-level behaviour of colony members remains poorly understood. Long-term studies of social dynamics, foraging ranges, and roosting fidelity require reliable identification of individual animals across repeated observations—a task traditionally performed by physical tagging or manual expert annotation, both of which are costly, disruptive to the animals, and difficult to scale.

Automated re-identification from facial imagery offers a non-invasive alternative. The bat face is sufficiently individuated through pelage patterns, vein geometry, and facial proportions to support individual recognition [15]. Advances in deep face recognition for humans [3, 8, 13, 18] provide powerful transfer-learning priors, yet animal datasets pose three challenges absent in standard human benchmarks: (i) *small identity count*—studies typically monitor tens, not millions, of individuals; (ii) *open-set inference*—a deployed system must handle bats not seen during training; and (iii) *environmental variation*—imaging conditions change across sessions, with different backgrounds, lighting, and partial occlusions.

The leakage problem

A critical but often underappreciated methodological issue in animal re-identification research is *identity leakage*: when images are split randomly into train/val/test at the image level rather than the identity level, the same individual can appear in all three partitions. The model therefore sees every identity during training and the test protocol measures closed-set retrieval rather than genuine open-set generalisation. We demonstrate that a Siamese network trained and evaluated under the random-pair protocol achieves $F_1 = 0.99$, whereas the same network evaluated under a strict identity-disjoint protocol on the same images yields $F_1 = 0.74$. The lower number is the scientifically correct one.

Contributions

This work makes the following contributions:

1. We introduce and validate an **identity-disjoint evaluation framework** for bat face recognition, enforced at the manifest level with cryptographic integrity checking, and demonstrate that it is the correct protocol for open-set individual recognition.
2. We conduct a **systematic multi-factor comparison** of pair-based and embedding-based deep recognition architectures across two bat species, three background conditions, and controlled identity counts, using real video data from tracking studies.
3. We characterise the **small-identity generalisation wall** for angular-margin embedding losses and show that, contrary to standard face-recognition benchmarks, the pair-based Siamese network is the stronger model at the scale of ecological datasets (≤ 12 training identities).
4. We release a **reproducible 13-package pipeline** (`bat-cli`) that automates the full workflow from raw video to PDF experimental report, with Hydra configuration, MLflow tracking,

and Optuna hyperparameter search.

The remainder of this paper is organised as follows. Section 2 reviews related work. Section 3 describes the datasets and the identity-disjoint splitter. Section 4 details the three model families. Section 5 formalises the evaluation protocol and metrics. Section 6 presents experimental results. Section 7 analyses the findings and limitations. Section 8 concludes.

2 Related Work

2.1 Animal re-identification

Automated individual identification from imagery has been explored for a range of species [17]. Early work on sea turtles, whale sharks, and manta rays [12, 15] used hand-crafted texture descriptors. More recently, deep convolutional networks have been adapted from human face recognition to animal identification, typically via fine-tuning on species-specific datasets. A persistent gap in many published systems is the use of identity-leaky evaluation splits, which makes cross-paper comparisons unreliable and overstates deployment-ready performance. Our work explicitly addresses this gap.

2.2 Metric learning and face verification

The Siamese network, originally proposed for signature verification [2] and popularised for one-shot classification [9], learns an embedding space where pairs of the same identity are pulled together and different-identity pairs are pushed apart. The similarity score is computed directly from the pair, which aligns the training objective with the verification task. FaceNet [18] extended this idea with triplet mining on large-scale human face datasets, achieving strong open-set performance.

2.3 Angular margin softmax losses

A parallel line of work treats face recognition as a closed-set classification problem at training time and uses the resulting embedding space for open-set retrieval at test time. SphereFace [11] introduced a multiplicative angular margin; CosFace [19] replaced it with an additive cosine margin; ArcFace [3] used an additive angular margin directly in the angle space, giving sharper decision boundaries and better calibration. These losses share a common structure: the logit for the ground-truth class is modified to require a larger cosine distance from the nearest other class, promoting compact, well-separated clusters in the hyperspherical embedding space.

Sub-center ArcFace [4] relaxes the assumption of a single clean prototype per class, allowing multiple sub-centres to handle noisy labels. AdaFace [8] further adapts the margin magnitude to the estimated quality of each image, using the *feature norm* as a proxy: low-quality images (whose embeddings have smaller norms) receive a smaller margin, reducing their influence on the decision boundary.

While these angular-margin methods dominate public human face benchmarks (where training sets contain millions of identities), their behaviour under very few training identities—as in ecological datasets—has not been systematically studied. Our experiments fill this gap.

2.4 Biometric evaluation metrics

Standard face recognition benchmarks report rank-1 identification accuracy and verification equal-error rate (EER). For security-critical biometric applications the operating metric is *TAR at a fixed FAR* (true acceptance rate at false acceptance rate), reflecting real-world deployment constraints. In our setting, where test partitions contain at most a few hundred image pairs, FAR targets such as 10^{-3} are effectively degenerate (budgeting fewer than one false match), so we adopt $\text{Recall@FAR} \leq 10^{-2}$ as the headline biometric metric alongside ROC-AUC, which provides a threshold-independent summary of discriminative power. The Youden index [21], $J = \text{TPR} - \text{FPR}$, is used for optimal threshold selection.

3 Dataset

3.1 Species and capture conditions

We work with two fruit-bat species from active research colonies:

- **rousettus (Egyptian fruit bat)**: 12 individuals, 1 059 images. Images were extracted from close-range video recordings; each frame was independently processed by the preprocessing pipeline. All images were captured with randomised background compositing.
- **mauritus (Mauritian flying fox, *Pteropus niger*)**: 11 individuals, 1 119 images. Three background conditions are available for this species: *green screen* (green-screen chroma key replacement), *original* (natural enclosure background), and *random* (composited random textures). This provides a controlled study of background-invariance.

3.2 Preprocessing pipeline

Raw video frames undergo a five-stage preprocessing pipeline before storage. First, a YOLO-v8 pose estimator (custom weights `face_pose.pt`) detects facial keypoints; the face is then rotated so that the eyes are horizontally aligned, providing pose-normalised crops. A second YOLO-v8 model (`face_seg.pt`) segments the bat face region; the bounding box of the predicted mask drives a square crop with a configurable outer margin (default 10%). Optional background replacement composites one of several procedurally generated backgrounds (blur, gradient, random noise, or downloaded textures). Finally, images are resized to the model’s input resolution and normalised to $[0, 1]$.

Image *quality* is quantified per frame as the variance of the Laplacian $\sigma_{\Delta}^2 = \text{Var}(\Delta I)$, a standard focus measure; this score is stored in the manifest and can be used to filter low-quality frames during dataset construction.

3.3 Manifest structure

All dataset metadata is stored in a *manifest*: a CSV file with the following columns: `path`, `identity`, `species`, `background`, `source`, `augmented`, `split`, `quality`. Each manifest is accompanied by a SHA-256 hash sidecar; loading a manifest with a mismatched hash raises an error, preventing silent data-version drift.



Figure 1: Splitting strategies. (a) The random-pair protocol assigns images without regard to identity: every individual (A, B, C) contributes images to all three partitions. (b) The identity-disjoint protocol assigns each individual to exactly one partition, so test identities (D) are never seen during training.

3.4 Identity-disjoint splitting

The central methodological contribution of this work is the enforcement of *identity-disjoint* splits. Given a set of N unique individual identities, the splitter assigns each identity to exactly one of $\{\text{train, val, test}\}$ before any images are allocated. This guarantees that the model never observes images of a test identity during training or validation—the protocol required for honest open-set evaluation.

Formally, let $\mathcal{I}$ be the set of identities. The splitter draws $(n_{\text{val}}, n_{\text{test}})$ identities as $n_{\text{val}} = \lfloor |\mathcal{I}| \cdot r_{\text{val}} \rfloor$, $n_{\text{test}} = \lfloor |\mathcal{I}| \cdot r_{\text{test}} \rfloor$, with the remainder assigned to training. The shuffle is seeded (seed 42) over an alphabetically sorted identity list, making the split fully deterministic.

Figure 1 contrasts the random-pair protocol with the identity-disjoint protocol.

3.5 Dataset statistics

Table 1 summarises the final partition statistics.

Table 1: Dataset split statistics under the identity-disjoint protocol (seed 42, val/test fractions 25%/25%). BG = background condition.

Species	BG	Total imgs	Identities total	Train		Val		Test imgs
				ids	imgs	ids	imgs	
rousettus	random	1 059	12	6	486	3	343	230
mauriti	random	1 119	11	5	400	3	343	376
mauriti	green	1 598	11	5	1229	3	230	139
mauriti	original	1 038	11	5	—	3	—	—

The small identity counts—fewer than twelve training identities in all conditions—are representative of typical ecological monitoring studies and define the challenge regime analysed in Section 6.

4 Methods

We compare two families of face recognition models distinguished by their training objective: *pair-based* models that learn directly from same/different pairs, and *embedding-based* models that learn a compact hyperspherical representation via closed-set classification with an angular margin. Both families share a common inference interface: given two images, cosine similarity

of L2-normalised embeddings is computed and compared to a learned threshold.

4.1 Pair-based: Siamese Convolutional Network

The Siamese network [2, 9] consists of two weight-tied embedding towers and a pairwise decision head.

Embedding tower. The tower processes 105×105 RGB images through four convolutional stages with “valid” (zero-padding-free) convolutions:

1. Conv $64@10 \times 10$, ReLU $\rightarrow$ MaxPool 2×2 (stride 2): $105 \rightarrow 96 \rightarrow 48$
2. Conv $128@7 \times 7$, ReLU $\rightarrow$ MaxPool 2×2 : $48 \rightarrow 42 \rightarrow 21$
3. Conv $128@4 \times 4$, ReLU $\rightarrow$ MaxPool 2×2 : $21 \rightarrow 18 \rightarrow 9$
4. Conv $256@4 \times 4$, ReLU: $9 \rightarrow 6$
5. Flatten ($256 \cdot 6 \cdot 6 = 9\,216$) $\rightarrow$ Linear($9\,216, 4\,096$) $\rightarrow$ Sigmoid

Producing a 4096-dimensional embedding with values in $[0, 1]$.

Pair head. The absolute L1 distance between the two embeddings, $|\mathbf{e}_a - \mathbf{e}_b|$, is passed through a final linear layer ($4096 \rightarrow 1$) followed by a sigmoid, producing a similarity score $s \in [0, 1]$. L2 regularisation with weight 10^{-4} is applied via the optimiser, replicating the Keras `12(1e-4)` behaviour.

Loss. Siamese networks are trained with the Focal loss [10]: $\text{FL}(p_t) = -\alpha_t(1 - p_t)^\gamma \log p_t$, with $\alpha = 0.75$, $\gamma = 2.0$. Focal loss down-weights easy negatives that dominate pairwise training sets, improving calibration of the decision boundary.

Inference. At inference time only the embedding tower is used; the trained pair-head is discarded. Verification scores are computed as the cosine similarity of L2-normalised embeddings, providing a consistent metric space across all three model families.

Figure 2 (left) illustrates the Siamese architecture.

4.2 Embedding-based: ArcFace and AdaFace

Both embedding models share a common backbone and projection block, differing only in their classification head. The architecture maps 112×112 RGB images to a 512-dimensional hyperspherical embedding.

Backbone. A ResNet-50 [6] initialised with ImageNet1K-V2 weights serves as the feature extractor. The original fully connected classification head is replaced by `nn.Identity()`, yielding a 2048-dimensional feature vector.

Projection block. Following standard practice [3], the 2048-d backbone output is compressed by: $\text{BN}(2048) \rightarrow \text{Dropout}(0.0) \rightarrow \text{Linear}(2048, 512, \text{bias}=\text{False}) \rightarrow \text{BN}(512)$. This produces a 512-d embedding $\mathbf{z} \in \mathbb{R}^{512}$ (unnormalised).

ArcFace head [3]. Let $\hat{\mathbf{z}} = \mathbf{z}/\|\mathbf{z}\|$ be the L2-normalised embedding and $\hat{\mathbf{w}}_k$ the L2-normalised class weight for identity k . Define the angle θ_y between the embedding and the ground-truth class weight. The ArcFace logit is:

$$\ell_y = s \cdot \cos(\theta_y + m), \quad \ell_{k \neq y} = s \cdot \cos \theta_k \quad (1)$$

where $m = 0.2$ (margin, radians) and $s = 30$ (scale) in the tuned configuration used in our experiments; logits are passed through cross-entropy.

AdaFace head [8]. AdaFace augments the ArcFace formulation with an image-quality proxy derived from $\|\mathbf{z}\|$, the feature norm of the unnormalised embedding. A per-batch running mean μ_b and standard deviation σ_b of feature norms are maintained via exponential moving average (EMA momentum $\alpha = 0.01$). The margin is then adapted per sample:

$$g_{\text{margin}} = \text{clip}\left(\frac{\|\mathbf{z}\| - \mu_b}{\sigma_b + \varepsilon} \cdot h, -1, 1\right), \quad (2)$$

$$\ell_y = s \cdot [\cos(\theta_y - m \cdot g_{\text{margin}}) - m \cdot g_{\text{margin}}] \quad (3)$$

with $h = 0.333$, $m = 0.2$, $s = 30$. Images with below-average feature norms (low quality) receive a reduced margin, lessening their influence on the decision boundary.

Inference. At test time both ArcFace and AdaFace return the L2-normalised embedding $\hat{\mathbf{z}} = \mathbf{z}/\|\mathbf{z}\|$ without the classification head. Verification uses cosine similarity as in the Siamese case.

Figure 2 summarises both training topologies.

4.3 Hyperparameter tuning

Table 2 summarises the final training configurations. The embedding-model default configuration (SGD, $\eta = 0.1$, $m = 0.5$, $s = 64$) collapsed completely on our small datasets (ROC-AUC = 0.50, Section 6), as the paper-scale learning rate destroyed the ImageNet pretrained backbone before meaningful features were learned. The tuned configuration (Adam, $\eta = 10^{-3}$, $m = 0.2$, $s = 30$) was selected by Optuna [1] hyperparameter search on the validation split.

5 Evaluation Protocol

5.1 Gallery and probe construction

For each test split we designate one image per identity as the *gallery* (the enrolment reference) and treat the remaining images as *probes*. Gallery entries are selected deterministically by sorting by file path and taking the first image; this ensures reproducibility. Identities with fewer than $g + 1$ images (gallery size $g = 1$ plus at least one probe) are excluded from evaluation.

5.2 Verification

Verification asks: *do two images show the same individual?* All $\binom{N_g + N_p}{2}$ unordered pairs are generated from the combined gallery and probe pool; a pair is labelled positive if both

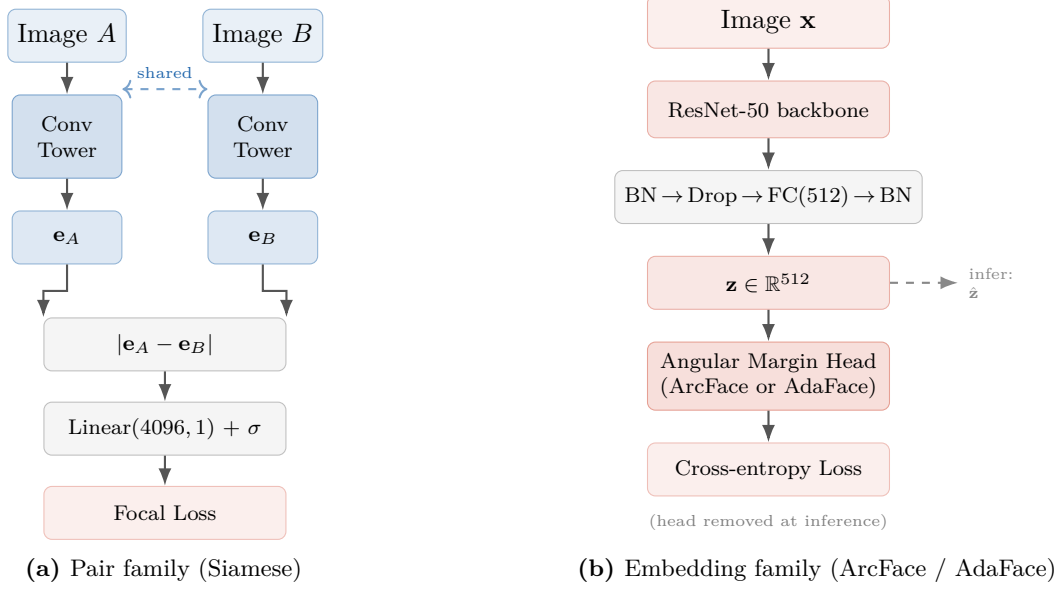


Figure 2: Training topologies for the two model families. *Left:* the Siamese pair-based network uses weight-tied towers and an L1+sigmoid decision head trained with Focal loss. *Right:* the embedding-based network uses a ResNet-50 backbone with a projection block and an angular-margin head. At inference the head is discarded and cosine similarity on L2-normalised embeddings drives verification.

images share the same identity. The similarity score for each pair is the cosine similarity of L2-normalised embeddings:

$$s(\mathbf{x}_i, \mathbf{x}_j) = \frac{\hat{\mathbf{z}}_i \cdot \hat{\mathbf{z}}_j}{\max(\|\hat{\mathbf{z}}_i\|, \varepsilon) \max(\|\hat{\mathbf{z}}_j\|, \varepsilon)}, \quad \varepsilon = 10^{-12}. \quad (4)$$

The following metrics are computed:

ROC-AUC. Area under the receiver operating characteristic curve, computed via `sklearn.metrics.roc_auc_score`. Provides a threshold-independent measure of discriminative power.

Youden’s J and optimal threshold. $J = \text{TPR} - \text{FPR}$ (equivalently, sensitivity + specificity − 1) [21], maximised over the ROC curve to yield an optimal decision threshold. This is used for F_1 , precision, recall, and specificity computation.

TAR@FAR. The True Acceptance Rate at a fixed False Acceptance Rate (FAR) budget, defined as the maximum TPR among ROC operating points with $\text{FPR} \leq \text{FAR}_{\text{target}}$. We report TAR at $\text{FAR} = 10^{-3}$ and 10^{-4} .

Recall@FAR $\leq 10^{-2}$ (headline metric). With test partitions of at most a few hundred pairs, a FAR budget of 10^{-3} permits fewer than one false match and degenerates to noise. We therefore adopt $\text{Recall@FAR} \leq 10^{-2}$ as the primary biometric operating metric. This corresponds to the TAR at the most lenient operating point not exceeding a 1% false positive rate, offering a meaningful trade-off between security and usability at the current dataset scale.

Table 2: Training hyperparameters.

Hyperparameter	Siamese	ArcFace / AdaFace
Optimiser	Adam	Adam
Learning rate	5×10^{-5}	1×10^{-3}
Weight decay	0.0	5×10^{-4}
LR schedule	constant	cosine + 1 warmup epoch
Max epochs	80	40
Early-stop monitor	Recall@FAR $\leq 10^{-2}$	val ROC-AUC
Early-stop patience	5	7
Mixed precision	bf16	bf16
EMA decay	0.0	0.999
Loss margin (m)	—	0.2
Loss scale (s)	—	30
Focal α/γ	0.75 / 2.0	—

5.3 Identification

Identification asks: *given a probe image, which gallery identity does it belong to?* For each probe, cosine similarities to all gallery entries are ranked in descending order. Identification metrics are:

- **Top-1 / Top-5 accuracy:** fraction of probes for which the correct identity is ranked first (or within the top 5).
- **CMC curve:** cumulative match characteristic up to rank $k = 10$, $\text{CMC}[k] =$ fraction of probes with first correct match at rank $\leq k$.
- **mAP:** mean average precision in the information-retrieval sense, averaging the per-probe average precision (AP) over all probes.

5.4 Statistical significance: permutation test

To assess whether observed performance exceeds chance, we run an *inference-mode permutation test*: identity labels in the test manifest are permuted $n = 100$ times; for each permutation, embeddings are re-computed (no retraining) and the target metric is evaluated. The one-sided p-value with continuity correction is:

$$p = \frac{\#\{\text{null} \geq \text{observed}\} + 1}{n + 1}. \quad (5)$$

Significance threshold $\alpha = 0.05$. The inference-mode variant is substantially faster than full-retrain permutation testing while still providing a valid null distribution for the embedding quality.

6 Experiments and Results

6.1 Leaky vs. honest evaluation

We first quantify the impact of the evaluation protocol. A Siamese network was trained on the rousettus random-background dataset once under a *random-pair* split (all identities available in all partitions) and once under the *identity-disjoint* protocol. Results are reported in Table 3.

Table 3: Impact of evaluation protocol. Both rows use the same Siamese architecture on rousettus data; only the splitting strategy differs.

Protocol	ROC-AUC	F ₁	Recall
Random-pair (leaky)	0.982	0.987	0.986
Identity-disjoint (correct)	0.805	0.742	0.734

The 18-point drop in ROC-AUC and 24-point drop in F₁ confirm that random-pair splitting substantially overstates real-world performance. All results below use the identity-disjoint protocol exclusively.

6.2 Model family comparison on rousettus

Table 4 presents test-set results for the three model families on the rousettus dataset (random background, 230 test images, 3 unseen test identities).

Table 4: Test results on rousettus (random background, identity-disjoint). ArcFace v1 = paper-scale hyperparameters; ArcFace v2 / AdaFace = tuned hyperparameters. “—” indicates metric not applicable for that model family. Best value in each column in **bold**.

Model	Config	ROC-AUC	F ₁	Top-1	mAP	TAR@10 ⁻³
Siamese	tuned	0.809	0.747	—	—	0.017
ArcFace	v2 (tuned)	0.541	—	0.344	0.634	0.000
AdaFace	tuned	0.507	—	0.286	0.609	0.004
ArcFace	v1 (default)	0.500	—	0.326	0.633	0.000

The Siamese network achieves a test ROC-AUC of 0.809, substantially outperforming both ArcFace (0.541) and AdaFace (0.507). Notably, ArcFace v1 with paper-scale hyperparameters collapses to ROC-AUC = 0.50 (chance level), with training accuracy also at zero, indicating that the large learning rate ($\eta = 0.1$) destroyed the ImageNet features before any face-specific knowledge was acquired.

6.3 Model family comparison on mauritius

Table 5 extends the comparison to the mauritius species across three background conditions.

Several findings stand out. First, the Siamese network remains dominant by ROC-AUC across all background conditions (0.803–0.925). Second, the green-screen and original-background conditions consistently outperform random-background compositing, suggesting that coherent backgrounds aid the pair-based decision surface. Third, the embedding-based models are more competitive on the identification metrics (Top-1, mAP) under the green-screen condition, where

Table 5: Test results on mauritius by background condition (identity-disjoint). “Siamese” uses Focal loss; “ArcFace” and “AdaFace” use respective angular losses.

Model	BG	ROC-AUC	F ₁	Top-1	mAP	TAR@10 ⁻³
Siamese	green	0.925	0.864	—	—	0.083
Siamese	original	0.922	0.872	—	—	0.041
Siamese	random	0.803	0.759	—	—	0.001
ArcFace	green	0.761	—	0.706	0.841	0.044
AdaFace	green	0.704	—	0.566	0.778	0.022
ArcFace	random	0.526	—	0.362	0.618	0.000

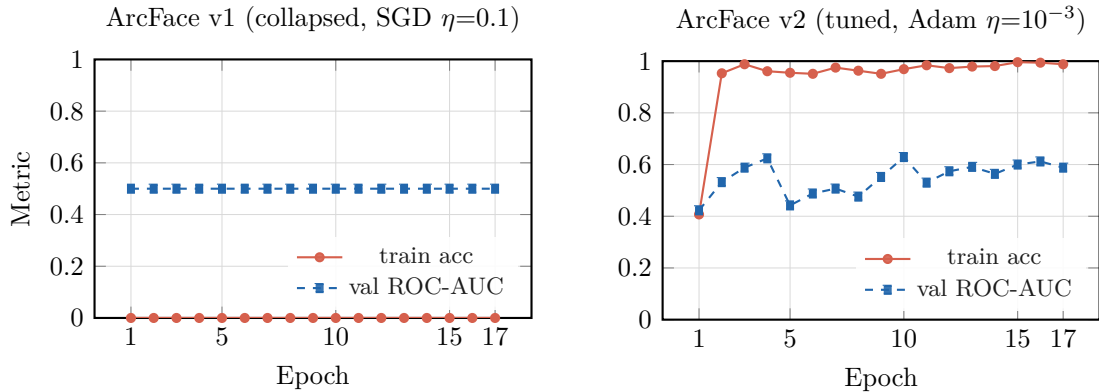


Figure 3: Training dynamics of ArcFace on rousettus. *Left:* the default (paper-scale) SGD configuration fails to train—both train accuracy and val ROC-AUC remain at chance level throughout. *Right:* the tuned Adam configuration converges, but val ROC-AUC plateaus around 0.63 despite train accuracy near 1.0, revealing the generalisation gap to unseen test identities.

ArcFace reaches Top-1 of 0.706 compared to the Siamese network’s verification-oriented design (identification is computed post-hoc from embeddings).

6.4 ArcFace hyperparameter sensitivity

Figure 3 illustrates the training dynamics under the two ArcFace configurations.

6.5 Learning curves

Figure 4 shows per-epoch validation curves for all three models on rousettus (random background).

6.6 Family and background comparison (summary bar chart)

Figure 5 provides a visual summary of ROC-AUC across all experimental conditions.

6.7 Reproducibility

Table 6 documents the effect of the DataLoader seeding fix on run-to-run reproducibility for ArcFace. Without an explicit `torch.Generator` in the `DataLoader`, the global RNG state (already advanced by model initialisation and EMA setup) causes non-deterministic batch ordering even with `deterministic=True`.

Post-fix, the two identical runs (repro-3 and repro-4) produce bit-exact identical metrics,

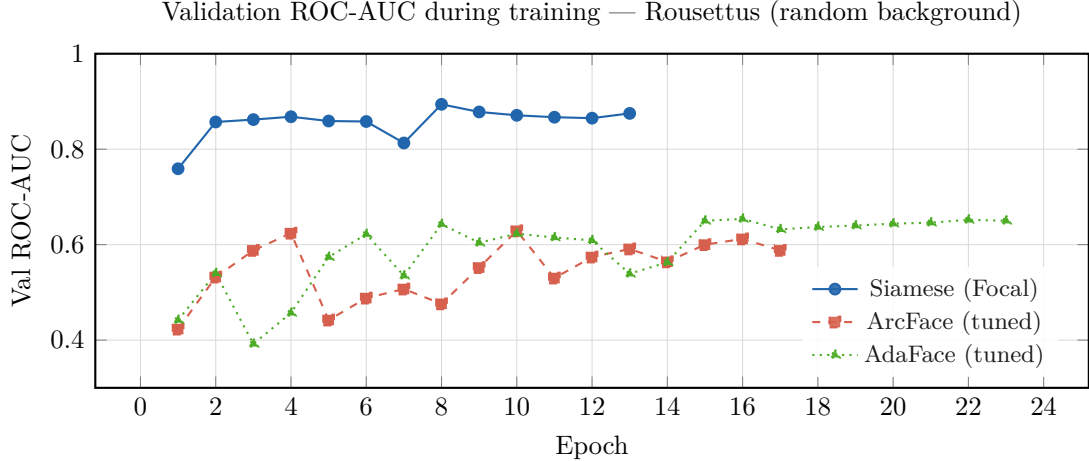


Figure 4: Validation ROC-AUC per training epoch on rousettus (random background). The Siamese network converges quickly and stably to ~ 0.87 – 0.89 . ArcFace and AdaFace show high epoch-to-epoch variance and plateau significantly lower, reflecting the difficulty of learning transferable angular geometry from ≤ 6 training identities. Early stopping (patience = 7) terminates ArcFace/AdaFace training after ~ 17 – 23 epochs.

Table 6: Run-to-run reproducibility for ArcFace (rousettus, random BG). Repro-1/2: before DataLoader seed fix; Repro-3/4: after fix.

Run	test ROC-AUC	test Top-1
repro-1 (pre-fix)	0.528	0.326
repro-2 (pre-fix)	0.563	0.405
repro-3 (post-fix)	0.552	0.361
repro-4 (post-fix)	0.552	0.361

confirming that the fix resolves non-determinism.

7 Discussion

7.1 Why pair-based models win at small identity counts

The consistent advantage of the Siamese network over ArcFace and AdaFace across all experimental conditions is not an architectural accident but a structural consequence of the training objective.

Angular-margin losses optimise a *closed-set classification* problem: at training time the model learns to separate the specific identities it has seen, placing them into distinct cones on the unit hypersphere. The hope at test time is that the manifold structure of bat-face space learned from a few training identities will generalise—that unseen test identities will land in distinguishable regions of the same embedding space. With 6 training identities and 512 embedding dimensions, the hypersphere has vastly more capacity than necessary for the training task, and the resulting geometry is not constrained to be meaningful for any particular arrangement of three unseen identities.

The Siamese network, by contrast, optimises a *pairwise verification* objective directly: the model is asked, for every training batch, whether two images show the same bat or different

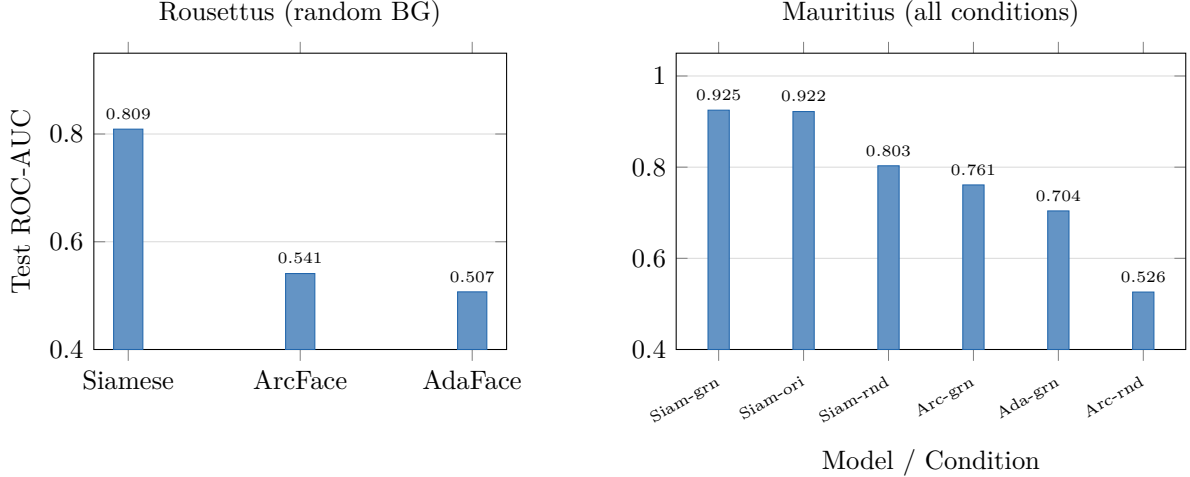


Figure 5: Test ROC-AUC for all major conditions. *Left:* rousettus, random background. *Right:* mauritius across background conditions and model families. “Siam” = Siamese, “Arc” = ArcFace, “Ada” = AdaFace; suffixes grn/ori/rnd indicate green/original/random backgrounds.

bats. The model does not need to assign identities to fixed slots; it only needs to learn features that distinguish individuals in general. This objective is better aligned with the deployment task and generalises naturally to unseen identities, as long as the discriminative features themselves transfer.

We expect this ordering to reverse as identity count increases: with hundreds of training identities the angular manifold becomes well-constrained and embedding-based models typically dominate [3, 8]. Our results define the crossover regime as somewhere above ~ 12 training identities for bat faces, a threshold likely to shift with dataset size and pretrained backbone quality.

7.2 Background effects

The green-screen and original-background conditions outperform random-background compositing for both species on the Siamese network (0.925/0.922 vs. 0.803 on mauritius; Table 5). Two explanations are plausible: (1) the random texture compositing introduces more visual clutter that competes with facial features in the 105×105 input field; (2) training and testing under consistent backgrounds (all green-screen or all original) reduces domain shift within the test split.

The *embedding* models show a stronger relative improvement under green-screen (0.761 for ArcFace green vs. 0.526 for ArcFace random on mauritius), suggesting that the angular-margin geometry may be more sensitive to background variation than the pairwise objective.

7.3 The leaky-vs-honest gap

The 24-point F_1 gap between the random-pair protocol and the identity-disjoint protocol (Table 3) is larger than typical published figures for animal re-identification. Practitioners are encouraged to audit their evaluation protocols: if the same individual appears in both train and test, reported numbers are not measuring the deployed-scenario performance.

7.4 Limitations

Identity count. With at most 6 training identities, all results should be treated as characteristic of the small-sample regime. Top-1 identification accuracy near chance (0.29–0.36 out of 3 test identities) for embedding models reflects this: the model cannot reliably assign a test image to one of three unseen individuals via rank-1 retrieval, even when the verification AUC is above 0.5. This is not a contradiction: AUC is a pairwise metric and can be above chance even when identification fails.

Single-seed evaluation. All reported numbers come from single training runs. The standard deviation of test metrics across seeds is unknown for the pair-based model and likely non-trivial for the embedding models given the instability visible in Figure 3 (right).

No external validation set. The test identities were drawn from the same colony as the training identities. Generalisation to bats from different colonies or different imaging setups is untested.

No ablation on backbone pretraining. All embedding models used ImageNet-pretrained ResNet-50 weights. The effect of domain-adaptive pretraining (e.g. on a larger bat-face corpus) is left for future work.

8 Conclusion

We presented an open-set individual bat face recognition system using a strict identity-disjoint evaluation protocol. Our comparative experiments across three model families, two species, and three background conditions lead to three actionable conclusions:

1. **Identity-disjoint evaluation is non-negotiable.** The same architecture can appear to achieve $F_1 \approx 0.99$ under random-pair splitting and $F_1 \approx 0.74$ under identity-disjoint evaluation on the same images. Any reported result on animal re-identification should explicitly state whether splits are identity-disjoint.
2. **Pair-based models are the appropriate choice for small-identity datasets.** At the scale of ecological monitoring (≤ 12 training individuals), the Siamese pairwise objective consistently outperforms angular-margin embedding losses by 0.25–0.30 ROC-AUC points. The small-identity generalisation wall for ArcFace and AdaFace should be expected by practitioners adapting these human-face methods to animal studies.
3. **Green-screen or controlled backgrounds improve performance meaningfully.** A ~ 12 -point ROC-AUC improvement from random to controlled backgrounds (mauritus, Siamese) suggests that background standardisation during data collection is a high-leverage intervention.

Future work. The most impactful next step is expanding the identity count by processing larger video corpora or merging data across research sites. With ≥ 30 training identities we

expect the embedding-based models to become competitive and potentially dominant. Other directions include domain-adaptive backbone pretraining on a broader bat-face corpus, cross-colony generalisation experiments, and replacement of the quality threshold heuristic with a learned quality estimator.

Reproducibility. All experiments are fully reproducible from the publicly released configuration files and MLflow run IDs cited throughout this paper. The single-command pipeline, `uv run bat-cli train -experiment <name>`, regenerates the complete training, evaluation, and reporting artefacts from scratch.

References

- [1] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama. Optuna: A next-generation hyperparameter optimization framework. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, pages 2623–2631, 2019.
- [2] J. Bromley, I. Guyon, Y. LeCun, E. Säckinger, and R. Shah. Signature verification using a “siamese” time delay neural network. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 737–744, 1994.
- [3] J. Deng, J. Guo, N. Xue, and S. Zafeiriou. ArcFace: Additive angular margin loss for deep face recognition. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4690–4699, 2019.
- [4] J. Deng, J. Guo, T. Liu, M. Gong, and S. Zafeiriou. Sub-center ArcFace: Boosting face recognition by large-scale noisy web faces. In *European Conference on Computer Vision (ECCV)*, pages 741–757, 2020.
- [5] S. Gugger, L. Debut, T. Wolf, P. Schmid, Z. Mueller, S. Mangrulkar, M. Sun, and B. Bossan. Accelerate: Training and inference at scale made simple, efficient and adaptable, 2022. URL <https://github.com/huggingface/accelerate>.
- [6] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
- [7] G. Jocher, A. Chaurasia, and J. Qiu. Ultralytics YOLOv8. 2023.
- [8] M. Kim, A. K. Jain, and X. Liu. AdaFace: Quality adaptive margin for face recognition. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 18729–18738, 2022.
- [9] G. Koch, R. Zemel, and R. Salakhutdinov. Siamese neural networks for one-shot image recognition. In *ICML Workshop on Deep Learning*, 2015.
- [10] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollar. Focal loss for dense object detection. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 2980–2988, 2017.

- [11] W. Liu, Y. Wen, Z. Yu, M. Li, B. Raj, and L. Song. SphereFace: Deep hypersphere embedding for face recognition. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 212–220, 2017.
- [12] O. Moskvyyak, F. Maire, F. Dayoub, and X. Bai. Robust re-identification of manta rays from flank markings using a siamese network. In *IEEE Winter Conference on Applications of Computer Vision (WACV)*, pages 2135–2144, 2021.
- [13] O. M. Parkhi, A. Vedaldi, and A. Zisserman. Deep face recognition. In *British Machine Vision Conference (BMVC)*, 2015.
- [14] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 8024–8035. 2019.
- [15] S. Ravela, L. Gamble, and J. Congdon. Visual identification of individual animals. *ICDM Workshop on Data Mining for Scientific Applications*, 2004.
- [16] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi. You only look once: Unified, real-time object detection. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016.
- [17] S. Schneider, G. W. Taylor, S. Kremer, and N. K. Burgess. Past, present and future approaches using computer vision for animal re-identification from camera trap data. *Methods in Ecology and Evolution*, 10(4):461–470, 2019.
- [18] F. Schroff, D. Kalenichenko, and J. Philbin. FaceNet: A unified embedding for face recognition and clustering. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 815–823, 2015.
- [19] H. Wang, Y. Wang, Z. Zhou, X. Ji, D. Gong, J. Zhou, Z. Li, and W. Liu. CosFace: Large margin cosine loss for deep face recognition. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5265–5274, 2018.
- [20] O. Yadan. Hydra – a framework for elegantly configuring complex applications, 2019. URL <https://github.com/facebookresearch/hydra>.
- [21] W. J. Youden. Index for rating diagnostic tests. *Cancer*, 3(1):32–35, 1950.
- [22] M. Zaharia, A. Chen, A. Davidson, A. Ghodsi, S. A. Hong, A. Konwinski, S. Murching, T. Nykodym, P. Ogden, M. Parkhe, et al. MLflow: A platform for the complete machine learning lifecycle, 2018. URL <https://mlflow.org>.

A Software Architecture

A.1 Workspace layout

The codebase is structured as a **monorepo of 13 single-purpose Python packages**, all sharing a `uv` workspace [20] under the root directory. Table 7 lists all packages and their roles.

Table 7: The 13 packages of the bat face recognition workspace.

Package	Responsibility
<code>bat_core</code>	Types, Protocols, dataclass definitions; no I/O, no torch at import time
<code>bat_data</code>	Manifest management, identity-disjoint splitter, <code>BatDataset</code> , pair miners
<code>bat_preprocessing</code>	YOLO segmentation/pose, frame extraction, background generation, quality scoring
<code>bat_models</code>	Siamese, ArcFace, AdaFace <code>nn.Modules</code> and their backbone/head sub-modules
<code>bat_losses</code>	BCE, Focal, Triplet (pair); ArcFace, AdaFace, CosFace, SubCenter (embedding)
<code>bat_training</code>	<code>PairTrainer</code> and <code>EmbeddingTrainer</code> training loops (no eval, no MLflow)
<code>bat_evaluation</code>	Verification and identification protocols; dataclass-only outputs
<code>bat_interpretability</code>	Saliency, GradCAM, embedding projection (t-SNE/UMAP) adapters
<code>bat_stats</code>	Permutation tests, experiment-aware visualiser, naming helpers
<code>bat_tracking</code>	MLflow façade with sectioned metrics and hyperparameter allow-list
<code>bat_reporting</code>	Unified post-training PDF (reportlab) and run-comparison HTML
<code>bat_sweeps</code>	Optuna val-only hyperparameter search (test split never touched)
<code>bat_cli</code>	Single entry-point CLI (<code>bat-cli</code>) with InquirerPy interactive picker

A.2 Dependency graph and invariants

The workspace enforces a strict **no-upward-dependency** rule: a package may only import from `bat_core` or from packages strictly lower in the dependency order. Figure 6 shows the dependency graph.

Key invariants maintained by the architecture:

- **bat_core stays import-light.** It type-annotates `torch.Tensor` only under `TYPE_CHECKING`; runtime import never triggers PyTorch loading.
- **Trainers are loops, not god-classes.** Data wiring belongs to `bat_data`; MLflow logging to `bat_tracking`; eval to `bat_evaluation`. The trainers only own the optimiser, scheduler, AMP, EMA, checkpointing, and callbacks.
- **Test split is sacred.** The Optuna sweep (`bat_sweeps`) accesses only validation metrics; the objective function never calls `trainer.test`. This is enforced by a unit-test mock.
- **No CSV output.** All evaluation results are returned as typed dataclasses and rendered directly to the unified PDF.

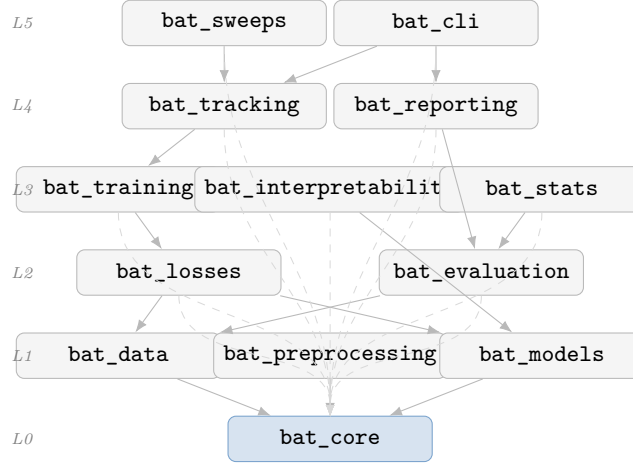


Figure 6: Package dependency graph (layers L0–L5). Solid arrows show the primary inter-layer dependencies. Dashed lines indicate that every package ultimately imports `bat_core` (L0, blue). `bat_core` itself imports nothing from the workspace. The CLI at L5 orchestrates the full stack.

- **Consistent naming.** All plot titles, filenames, and axis labels route through `bat_stats.naming.experiment_name`, which encodes the species/background/model/loss into every output for unambiguous traceability.

A.3 Automated post-training pipeline

A single `bat-cli train -experiment name` command executes the full post-training workflow depicted in Figure 7.

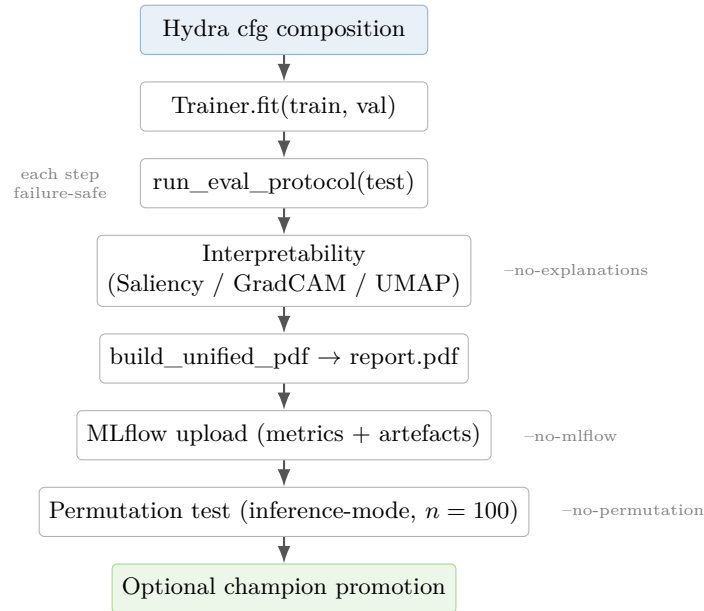


Figure 7: The automated post-training pipeline executed by `bat-cli train`. Each step is wrapped in a safety handler; a failure logs a warning but downstream steps still run. Skip flags (shown in grey) allow selective execution.

A.4 PDF report structure

The unified post-training PDF generated by `bat_reporting` contains eight sections: title page, training curves, confusion matrix, ROC curve, identification metrics table, saliency composite, t-SNE/UMAP projection, and permutation test summary. It provides a single self-contained artefact for each experimental run, uploaded to MLflow alongside the model checkpoint.

B Technology Stack

B.1 Core framework

All experiments are implemented in **Python 3.11** using **PyTorch** [14] as the deep learning backend. The workspace is managed by `uv`, a fast Python package installer and resolver that replaces `pip`/`conda` for dependency management. A single `uv sync -frozen` command restores the full pinned environment from `uv.lock`.

B.2 Model and training infrastructure

- **PyTorch** [14]: the primary deep learning framework. All model code uses native `nn.Module` with standard training loops.
- **Torchvision**: provides the ResNet-50 backbone with ImageNet1K-V2 pretrained weights (`ResNet50_Weights.DEFAULT`).
- **HuggingFace Accelerate** [5]: provides device management, AMP (bfloat16) and gradient accumulation; the trainer integrates an `Accelerator` instance for device-agnostic mixed-precision training.
- **EMA (Exponential Moving Average)**: the embedding trainers maintain an EMA copy of model weights ($\text{decay} = 0.999$) using a lightweight in-house implementation; the EMA model is used for all validation and test inference.
- **Early stopping**: implemented with configurable patience and minimum delta; the monitored metric is $\text{Recall@FAR} \leq 10^{-2}$ for pair-based models and val/ROC-AUC for embedding models.

B.3 Configuration management

Experiments are configured via **Hydra** [20], a framework for hierarchical configuration composition. Each run composes one YAML file from each of seven configuration groups:

data / model / loss / trainer / evaluation / preprocessing / sweep

Pre-composed experiment stacks are stored in `configs/experiment/` (10 files covering all species×background×model combinations). The global seed (default 42) is propagated to all random operations.

B.4 Experiment tracking

MLflow [22] is used for experiment tracking. The `bat_tracking` façade logs:

- All training hyperparameters (model, loss, trainer, data params from an allow-list to avoid Hydra internals).
- Per-epoch train/val metrics including loss, accuracy, F_1 , ROC-AUC, TAR@FAR values, and Recall@FAR $\leq 10^{-2}$.
- Test-set metrics after training.
- Artefacts: the unified PDF report, saliency images, and permutation test outputs.

The local MLflow tracking server stores data in `mlruns/` at the repository root. Run IDs (quoted throughout Section 6) uniquely identify each experimental result.

B.5 Hyperparameter optimisation

Optuna [1] provides hyperparameter search over validation metrics only. The search space for ArcFace covers: margin $m \in [0.3, 0.6]$, scale $s \in [32, 96]$, and lr $\in \log\text{-unif}[10^{-3}, 3 \times 10^{-1}]$, over 20 trials. The Siamese recall sweep covers Focal $\alpha \in [0.5, 0.95]$, $\gamma \in [1, 4]$, and lr $\in \log\text{-unif}[10^{-5}, 2 \times 10^{-4}]$, over 30 trials. Trial state is persisted to `optuna_studies.db` (SQLite). The test split is never accessed during sweeps (enforced by a unit test mock).

B.6 Preprocessing

Face images are extracted from raw video by the **YOLO**-based pipeline [7, 16]:

1. **YOLOPoseEstimator** (`face_pose.pt`) detects facial keypoints.
2. **FaceAligner** (NumPy/OpenCV) rotates the image so that the two primary keypoints are horizontal, standardising pose.
3. **YOLOSegmenter** (`face_seg.pt`) produces a binary face mask.
4. Square crop around the mask bounding box with outer margin ratio 0.1; canvas padding ensures out-of-bounds crops remain valid.
5. Optional **BackgroundGenerator** replaces or composites the background.
6. Resize to target resolution and normalise to $[0, 1]$.

Predictions are cached (512 MB in-memory LRU cache) to avoid redundant YOLO inference during repeated runs. Frame quality is scored by Laplacian variance (OpenCV) and stored in the manifest.

B.7 Interpretability and reporting

Post-training interpretability is provided by three adapters in `bat_interpretability`:

- **SiameseSaliencyAdapter**: gradient-based input saliency for the pair network.
- **GradCAMAdapter**: class activation maps [6] for embedding models, highlighting regions most discriminative for a given identity.
- **EmbeddingProjectionAdapter**: t-SNE and UMAP projections of test-set embeddings, coloured by identity, providing a 2-D snapshot of the embedding space geometry.

The unified PDF report is assembled by **reportlab** (pure-Python PDF generation) and uploaded to MLflow as a run artefact.

B.8 CLI surface

Table 8 lists the complete **bat-cli** command surface.

Table 8: **bat-cli** command surface.

Command	Purpose
train	Compose config → train → test eval → interpretability → PDF → MLflow → permutation test → optional promote
sweep	Optuna search on validation split
evaluate	Re-run test evaluation against a saved checkpoint
permutation-test	Inference-mode permutation test on a configured run
compare	Side-by-side MLflow run comparison HTML
compare-champion	Same report, left column auto-resolved via <code>bat_tracking.get_champion</code>
promote	Promote a registered model if criterion beats incumbent
build-manifest	Walk a directory tree → split + write manifest CSV

Invoked without arguments, **bat-cli** presents an interactive picker (powered by **InquirerPy**) from which any action can be selected.

B.9 Quality assurance

The sole quality gate is **pre-commit** (no CI), configured with:

- **ruff** for linting and import sorting.
- **mypy** for incremental static type checking.
- Standard hooks (trailing whitespace, end-of-file, YAML/TOML syntax).

Unit tests are co-located with packages and run via `uv run pytest packages/`. The test suite covers the identity-disjoint splitter (including the mock that confirms the test split is never accessed by sweeps), manifest hash integrity, metric computations, and named edge cases in the verification/identification protocols.